

AGV 工况感知与 LPV-MPC 闭环控制仿真软件 V1.0 软件设计

说明书

软件全称：AGV 工况感知与 LPV-MPC 闭环控制仿真软件

软件简称：AGV-MTCN-MPC 仿真软件

版本号：V1.0

生成日期：2026-05-25

编制说明：本文档根据当前仓库源码、数据契约、项目流程清单和默认部署配置生成，用于软件著作权登记辅助材料。申请人应在提交前确认软件名称、著作权人、开发完成日期、首次发表日期和权属证明。

修订记录

版本	日期	内容	责任人
V1.0	2026-05-25	根据当前项目生成软著 申请辅助设计说明书	[申请人填写]

目录

第 1 章 软件概述
第 2 章 运行环境
第 3 章 总体架构
第 4 章 数据与路径生成模块
第 5 章 AGV 模型与 LPV-MPC 控制模块
第 6 章 ModernTCN 工况感知模块
第 7 章 训练、评估与模型导出
第 8 章 Simulink 闭环仿真模块
第 9 章 对照算法与基线模块
第 10 章 实验、报告与验证模块
第 11 章 用户使用说明
第 12 章 技术特点
第 13 章 数据安全性与维护
第 14 章 版本说明
附录 A 主要源码文件清单
附录 B 主要输入输出文件清单
附录 C 关键参数表
附录 D 术语表

第 1 章 软件概述

1.1 软件名称、简称与版本

本软件全称为“AGV 工况感知与 LPV-MPC 闭环控制仿真软件”，简称为“AGV-MTCN-MPC 仿真软件”，版本号为“V1.0”。本说明书、源码页眉、申请字段草稿和一致性检查清单均以该名称和版本为统一标识。

软件名称中的“AGV 工况感知”对应 ModernTCN 多任务时序模型；“LPV-MPC 闭环控制”对应 AGV 车辆模型、LPV 线性化网格、MPC 控制器和在线调度更新；“仿真软件”对应 MATLAB/Simulink 桌面闭环验证平台。

1.2 开发背景

本软件面向对角双转向驱动 AGV 的工况识别、坡度调度和闭环控制仿真需求。AGV 在直行、转向、坡道、低速堵转和扰动条件下的传感量具有时序相关性，单点规则或固定参数控制难以完整描述实际工况变化。

项目通过 MATLAB/Simulink 建立车辆模型和 LPV-MPC 控制平台，通过 Python/PyTorch 训练 ModernTCN 多任务时序感知模型，再通过 ONNX 和 MATLAB 侧加载器接入闭环仿真。该流程支持从离线数据构建到在线控制验证的完整工程链路。

1.3 软件目标

本软件目标是提供一套可复现的 AGV 工况感知与闭环仿真工具链。软件能够生成参考路径和训练路径，构建 128 步、19 维输入窗口，训练 ModernTCN 工况感知模型，导出 ONNX 模型，并在 Simulink 闭环模型中调用预测结果辅助 LPV-MPC 调度。

软件同时提供 GRU、TCN、LPV-MPC θ_0 、IMU θ 和 true- θ oracle 等对照链路，用于验证 ModernTCN 感知输出对闭环轨迹跟踪、控制平滑性、鲁棒性和实时性的影响。

1.4 适用对象与应用场景

软件适用于 AGV 控制算法研究、工况感知模型验证、LPV-MPC 调度策略仿真、路径跟踪闭环性能比较和多算法公平对照实验。典型用户包括控制算法开发人员、仿真平台维护人员和研究型项目成员。

当前版本主要用于桌面仿真和算法验证，不直接声明为嵌入式控制器固件。ONNXRuntime+MPC 核心链路可用于评估计算余量；MATLAB/Simulink extrinsic 封装主要用于闭环仿真验证和调试。

1.5 软件边界

软件边界包括项目初始化、车辆模型、LPV-MPC、路径生成、数据集构建、ModernTCN 训练与部署、MATLAB 在线推理、Simulink 闭环仿真、对照算法和测试验证。软件不包含第三方深度学习框架源码、MATLAB/Simulink 平台源码、训练数据权属证明或申请人身份证明材料。

`.mat`、`.pt`、`.onnx`、`.slx` 等文件作为数据、模型、部署产物或工程模型在说明书中引用，不作为本次源程序文本鉴别材料纳入。

第 2 章 运行环境

2.1 硬件环境

软件可在普通 PC 工作站上运行，建议使用多核 CPU 和 16GB 及以上内存。若进行 ModernTCN、GRU 或 TCN 训练，可选 NVIDIA GPU 以缩短训练时间；若仅执行已训练模型的闭环仿真和结果分析，CPU 环境也可运行。

实时性测试脚本将 ONNXRuntime 单窗口推理和 MPC 求解时间作为核心链路进行测量，控制采样周期为 $T_s=0.01\text{ s}$ 。测试结果用于评价计算余量，不等同于对所有桌面仿真封装开销的硬实时承诺。

2.2 软件环境

软件环境包括 Windows 10/11 或兼容桌面操作系统、MATLAB/Simulink、Python、PyTorch、ONNXRuntime，以及必要的 MATLAB 工具箱。当前仓库中 MATLAB 代码用于车辆模型、控制器、路径和闭环仿真；Python 代码用于 ModernTCN 训练、ONNX 导出和 ONNXRuntime 一致性测试。

Simulink 主模型包括 ``simulink/LPVMPC_AGV_simulink_Modern_TCN.slx``、``simulink/LPVMPC_AGV_simulink_GRU.slx``、``simulink/LPVMPC_AGV_simulink_TCN.slx`` 和 ``simulink/LPVMPC_AGV_simulink_IMU.slx``。这些模型存在于当前仓库中，作为闭环仿真平台使用。

2.3 开发语言与依赖

软件主体源代码由 MATLAB 和 Python 编写。MATLAB 部分包括 ``src/core``、``src/lpv``、``src/mpc``、``src/paths``、``src/ModernTCN`` 中的在线推理封装、``src/gru``、``src/TCN`` 和 ``src/Compare``；Python 部分主要包括 ``src/ModernTCN/*.py`` 和 ``src/Compare/benchmark_modern_tcn_onnx_runtime.py``。

PyTorch、ONNXRuntime、MATLAB/Simulink 作为运行环境或依赖工具，不作为申请人的自有源代码。软件说明书仅描述本仓库围绕 AGV 场景实现的工程流程、数据契约和闭环仿真功能。

2.4 输入输出文件环境

核心输入包括车辆参数、路径 `.mat` 文件、统一数据集、scaler、LPV 网格模型、MPC 控制器和已导出的 ONNX 模型。当前 ModernTCN 主线数据集为

``data/tcn/ModernTCN_dataset_agv_dualsteer_theta10_uniform_conf_h0_v2.mat``，数据契约为 ``data/tcn/ModernTCN_dataset_agv_dualsteer_theta10_uniform_conf_h0_v2_contract.json``。

核心输出包括训练指标、ONNX 文件、一致性检查报告、闭环仿真 ``logout``、对比指标 CSV、鲁棒性报告、实时性报告和论文图表。运行结果主要存放在 ``results/``，数据和模型主要存放在 ``data/``。

2.5 目录结构说明

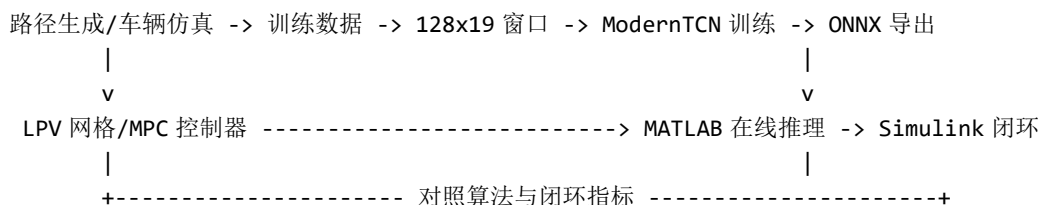
仓库根目录包含 `init\_project.m`、`project\_root.m` 和 `results\_dir.m` 作为路径管理入口。`src/core` 提供车辆模型和控制基础层，`src/lpv` 和 `src/mpc` 提供 LPV-MPC 控制模块，`src/paths` 提供路径生成模块，`src/ModernTCN` 提供主感知模型，`src/Compare` 提供闭环对比和实时性测试。

`data/` 目录保存训练数据、路径数据和模型缓存；`results/` 目录保存训练与闭环结果；`simulink/` 目录保存闭环模型；`docs/` 与论文相关目录只作为辅助说明和历史材料，不纳入本次源程序鉴别材料。

第 3 章 总体架构

3.1 系统总体架构

软件采用“离线训练 + 在线闭环仿真”的总体结构。离线阶段生成路径和训练数据，构建统一时序窗口，训练 ModernTCN 多任务模型并导出 ONNX；在线阶段由 MATLAB/Simulink 加载车辆模型、LPV-MPC 控制器和 ModernTCN 预测器，在每个控制周期更新工况、转向和坡度调度量。



3.2 功能模块划分

软件划分为项目初始化、车辆模型、LPV 线性化、MPC 控制、路径生成、数据集构建、ModernTCN 训练、ModernTCN 部署、Simulink 闭环、对照实验和测试验证模块。各模块通过固定文件路径和数据契约连接，避免不同算法使用不同数据或不同控制平台造成对比偏差。

核心控制模块为 `src/core`、`src/lpv` 和 `src/mpc`；核心感知模块为 `src/ModernTCN`；核心对比验证模块为 `src/Compare`。GRU 和 TCN 位于 `src/gru` 与 `src/TCN`，在说明书中作为对照和验证模块描述。

3.3 数据流与控制流

数据流从路径生成和仿真输出开始，经过训练数据构建、窗口化、归一化和数据划分后进入 ModernTCN 训练。模型输出包括主工况 logits、转向 logits 和坡度回归值，后续通过 ONNX 文件传递到 MATLAB 端。

控制流由 Simulink 闭环模型驱动。每个仿真步中，AGV 模型输出传感和状态量，ModernTCN 在线模块维护历史窗口并完成预测，LPV-MPC 根据调度量更新控制器，控制输入再作用于车辆模型。

3.4 离线训练与在线闭环关系

离线训练使用统一数据集和固定 split，避免窗口泄漏。在线闭环使用同一 scaler 策略，将实时观测量按训练集统计量归一化后送入 ModernTCN。该设计使训练、测试和闭环部署遵循同一数据契约。

训练阶段输出的 checkpoint、ONNX、summary CSV 和一致性报告用于冻结模型。默认部署模型为 seed 21，对应 run_tag `modern\_tcn\_theta10\_uniform\_h0\_v2\_seed21` 和 ONNX 文件 `results/modern\_tcn/modern\_tcn\_theta10\_uniform\_h0\_v2\_seed21/modern\_tcn\_seed21.onnx`。

3.5 Simulink 与 MATLAB/Python 协同方式

Python 负责 ModernTCN 模型定义、训练、指标计算和 ONNX 导出。MATLAB 负责加载 ONNX、组织在线窗口、进行闭环仿真、更新 LPV-MPC 控制器并保存仿真结果。Simulink 模型通过预加载函数和 MATLAB Function 包装连接这些模块。

该协同方式保留了 Python 深度学习训练效率和 MATLAB/Simulink 控制仿真能力。当前 MATLAB/Simulink 调用方式主要用于桌面仿真验证，不作为嵌入式部署实现的直接声明。

第 4 章 数据与路径生成模块

4.1 参考路径生成

路径生成模块位于 `src/paths`。基础路径接口 `gen\_agv\_ref\_path.m` 提供 ref 结构，`gen\_agv\_theta10\_uniform\_paths.m` 生成 theta10 uniform 训练路径，`gen\_factory\_logistics\_showcase\_path.m` 生成工厂物流展示路径，`gen\_closed\_loop\_eval\_paths.m` 生成多路径闭环评估路径。

路径结构包含时间、位置、航向、速度、曲率、坡度或调度相关元数据。路径生成模块同时保存预览图和报告，便于检查路径长度、转向段、坡道段和评估覆盖范围。

4.2 训练数据生成

训练数据由 AGV 仿真模型和参考路径共同生成。`src/core/agv\_model\_sfunc\_train\_data.m`、`state\_eq\_ref\_train\_data.m` 和 `output\_eq\_ref\_train\_data.m` 用于训练数据仿真链路，输出可观测传感量、车辆状态和标签所需信息。

当前主线 raw train data 为

`data/tcn/ModernTCN\_train\_data\_agv\_dualsteer\_theta10\_uniform\_conf\_h0\_v2.mat`，窗口化后的统一数据集为 `data/tcn/ModernTCN\_dataset\_agv\_dualsteer\_theta10\_uniform\_conf\_h0\_v2.mat`。该数据集被 ModernTCN、GRU 和 TCN 共同使用。

4.3 数据窗口化

窗口化策略固定为 `seq\_len=128`，控制采样周期为 `Ts=0.01 s`，因此单个输入窗口覆盖约 1.28 秒历史观测。窗口化后的输入形状为 `[window, time, feature]`，Python 训练时进入模型前再转换为卷积所需形式。

窗口标签采用 `current\_window\_end` 策略，即以当前窗口末端对应工况作为主工况、转向方向和坡度调度量标签。数据契约中 `horizon\_steps=0`，表示当前默认版本不做未来步预测。

4.4 数据契约

数据契约文件

`data/tcn/ModernTCN\_dataset\_agv\_dualsteer\_theta10\_uniform\_conf\_h0\_v2\_contract.json` 是当前数据链的权威定义。契约记录 `vehicle\_type=diagonal\_dual\_steer\_drive\_agv`、主动驱动/转向轮为 `LF` 和 `RR`、被动支撑轮为 `RF` 和 `LR`。

契约同时记录 `split\_policy=run\_level\_no\_window\_leakage` 和

`scaler\_policy=fit\_train\_only\_apply\_val\_test\_online`。前者用于降低同一运行片段窗口泄漏风险，后者用于避免验证集、测试集或在线数据参与 scaler 拟合。

4.5 19 维输入特征

当前 ModernTCN 输入维度为 `input\_dim=19`。19 个输入特征依次为：

```
accel_x
gyro_z
I_lf
I_rr
omega_wheel_lf
omega_wheel_rr
delta_lf
delta_rr
gyro_y
v_hat
dv_hat_dt
ws_imbalance
I_sum
I_diff_signed
I_diff_abs
accel_x_lp
kappa_proxy
accel_per_current
pitch_angle_est
```

这些特征覆盖纵向加速度、偏航角速度、左右驱动电流、车轮角速度、转向角、俯仰相关量、速度估计、电流组合量、曲率代理量和坡度估计量，满足工况识别、转向识别和坡度回归的输入需求。

4.6 主工况、转向方向与坡度标签

主工况分类标签为 `flat=1`、`stall=2`、`slope=3`。转向方向分类标签为 `right=-1`、`straight=0`、`left=1`。

坡度回归输出为 `theta\_hat`，用于闭环中的坡度感知或调度参考。

ModernTCN 输出为三组结果：`logits\_main` 是 3 维主工况分类 logits，`logits\_turn` 是 3 维转向方向分类 logits，`theta\_hat` 是 1 维坡度或调度量回归输出。

第 5 章 AGV 模型与 LPV-MPC 控制模块

5.1 AGV 车辆模型

AGV 模型面向 diagonal dual steer drive AGV。主动驱动和转向轮为 `LF` 与 `RR`，被动支撑轮为 `RF` 与 `LR`。状态向量包含位置、航向、速度、偏航角速度、左右转向角和侧偏相关状态。

`src/core/state\_eq.m` 实现非线性车辆状态更新，`src/core/output\_eq.m` 实现车辆输出和传感量计算。训练数据链路使用对应的 `train_data` 版本，以便生成与在线工况感知一致的观测特征。

5.2 状态方程和输出方程

状态方程根据当前状态、控制输入、车辆参数、转向几何、轮胎侧向力、滚动阻力和坡度阻力等计算下一步状态。采样周期由 `parameters.m` 中的 `params.Ts = 0.01` 定义。

输出方程生成基础状态、转向角、电流、轮速、IMU 相关量和用于深度模型输入的原始观测。输出方程中明确 RF/LR 为万向支撑轮或被动支撑轮，侧向和偏航动力主要由 LF/RR 舵驱轮承担。

5.3 参考模型

参考模型由 `state\_eq\_ref.m`、`output\_eq\_ref.m` 和训练数据参考版本组成。参考模型根据路径和目标速度生成可跟踪的参考状态与输出，为 LPV-MPC 和训练数据仿真提供一致的参考轨迹。

参考路径和参考模型共同定义直行、转向、坡道和工况切换过程，使数据集覆盖多种闭环验证场景。说明书中引用参考模型时，仅描述其软件功能，不将其写成论文推导。

5.4 LPV 线性化

LPV 线性化模块位于 `src/lpv`。`lin\_agv\_at\_point.m` 对 AGV 在指定状态点进行线性化，`lin\_agv\_grid.m` 生成网格化线性模型数据库。线性化结果供 MPC 控制器创建和在线调度更新使用。

LPV 网格模型的作用是将非线性车辆模型在不同调度点近似为一组线性模型。在线运行时，控制器可根据调度量选择或插值相应模型，提高不同工况下的控制适应性。

5.5 MPC 控制器构造

MPC 模块位于 `src/mpc`。`mpc\_setup\_single\_interp.m` 负责构造控制器，`mpc\_update\_from\_rho.m` 根据调度量更新控制模型，`Cost\_Function.m` 计算控制代价。控制器与车辆模型、参考路径和调度量共同构成闭环。

控制器目标包括轨迹跟踪、控制输入平滑和约束处理。ModernTCN 输出的坡度或调度量不是单独完成控制，而是作为 LPV-MPC 闭环调度的一部分使用。

5.6 在线调度更新

在线调度更新由 ModernTCN 预测、LPV 模型选择和 MPC 参数更新共同完成。预测器输出 ``theta_hat`` 后，MATLAB 端根据默认配置中的增益、限幅、变化率限制和 `deadzone` 等参数进行调理。

``ModernTCN_default_config.m`` 中包含 ``theta_output_gain``、``theta_abs_limit``、``theta_rate_limit`` 和 ``theta_mpc_deadzone`` 等部署侧调理参数。该设计用于避免单步预测噪声直接造成控制器剧烈变化。

5.7 控制代价与约束处理

控制代价函数用于评价轨迹误差、控制输入、输入变化和约束触碰等因素。闭环对比脚本进一步统计横向误差、航向误差、控制平滑性、坡度误差和综合排序指标。

说明书中可描述闭环结果和实时性结果，但应避免将桌面仿真结果夸大为已经完成嵌入式硬实时部署。当前更准确的表述是：ONNXRuntime+MPC 核心链路满足 10 ms 控制周期的计算余量，MATLAB/Simulink 封装主要用于仿真验证。

第 6 章 ModernTCN 工况感知模块

6.1 模块定位

ModernTCN 是当前软件主感知模块，位于 ``src/ModernTCN``。该模块面向 AGV 闭环控制的工况感知需求，将 128 步、19 维可观测特征映射为主工况分类、转向方向分类和坡度调度量回归。

ModernTCN 不直接替代 LPV-MPC 控制器，而是向控制器提供工况和调度辅助信息。该定位使软件能够在同一控制平台上比较不同感知模型对闭环控制效果的影响。

6.2 输入输出定义

输入为 ``[batch, time=128, feature=19]`` 的归一化窗口。输出为 ``logits_main``、``logits_turn`` 和 ``theta_hat``。ONNX 导出脚本中明确输出名称为 ``logits_main``、``logits_turn``、``theta_hat``。

分类 logits 后续可通过 softmax 转换为置信度和类别预测；坡度输出经过部署侧调理后作为 LPV-MPC 调度参考。该输出契约与数据契约中的

``output_contract=logits_main3_logits_turn3_theta1_with_softmax_confidence`` 一致。

6.3 ModernTCN-small 网络结构

当前默认 ModernTCN 部署模型使用 seed 21，配置包括 ``channels=64``、``blocks=5``、``kernel_size=31``、``temporal_padding=same``、``dropout=0.15``、``expansion=2``。模型定义位于 ``modern_tcn_model.py``。

模型首先使用 1x1 卷积将 19 维输入映射到通道空间，然后堆叠大核 depthwise temporal convolution 残差块，再通过时序池化和窗口统计特征融合形成任务头输入。

6.4 大核深度卷积残差块

ModernTCN 残差块使用 **depthwise temporal convolution** 捕获长时间窗口内的局部和中程时序模式，再通过 **pointwise channel mixing** 实现通道间信息融合。残差连接和 **layer scale** 用于稳定训练和导出。

当前默认模型使用 `'same'` padding，保证输出时间长度与输入窗口一致。项目中另有 **causal ModernTCN** 消融实验，但不替代默认 **seed 21** 部署模型。

6.5 多任务输出头

多任务输出头包括主工况分类头、转向方向分类头和坡度回归头。主工况分类头输出 3 维 **logits**，对应 **flat**、**stall**、**slope**；转向分类头输出 3 维 **logits**，对应 **right**、**straight**、**left**；坡度头输出 1 维 `'theta_hat'`。

多任务结构使软件能同时提供离散工况状态和连续调度量。闭环控制使用这些输出时，会结合限幅、变化率限制和 **deadzone** 处理，避免将模型预测直接无限制地作用到控制器。

6.6 窗口统计特征融合

模型在时序卷积特征之外融合窗口统计量，包括末端值、均值、最大值、最小值或其它输入统计特征。该设计用于增强模型对窗口整体趋势和末端状态的表达能力。

转向任务还使用特定特征索引相关统计量，例如偏航角速度、轮速、转向角、速度变化和曲率代理量，以提高转向方向及过渡窗口识别能力。

6.7 坡度输出调理与调度约束

坡度输出在模型内可根据主工况概率进行 **gate** 处理，部署侧还通过 `'theta_abs_limit'`、`'theta_rate_limit'` 和 `'theta_mpc_deadzone'` 控制调度量范围与变化速度。该处理位于模型定义和 MATLAB 默认配置共同作用的链路中。

调度约束的目的不是掩盖模型误差，而是使闭环控制对预测噪声更加稳健。说明书中应将其描述为工程部署侧保护和调理机制。

6.8 ONNX 导出与部署

`'export_modern_tcn_onnx.py'` 将训练好的 checkpoint 导出为 ONNX，并保存 PyTorch 参考输出。默认 ONNX 文件为

`'results/modern_tcn/modern_tcn_theta10_uniform_h0_v2_seed21/modern_tcn_seed21.onnx'`，输入形状为 `'[1,128,19]'`，输出名称为 `'logits_main'`、`'logits_turn'`、`'theta_hat'`。

`'check_onnxruntime_consistency.py'` 和 `'ModernTCN_check_matlab_onnx.m'` 分别用于 ONNXRuntime 与 MATLAB 端一致性检查。该设计使 PyTorch 训练结果、ONNX 部署文件和 MATLAB 闭环调用之间可追溯。

第 7 章 训练、评估与模型导出

7.1 训练入口

ModernTCN 单 seed 训练入口为 ``src/ModernTCN/train_modern_tcn.py``，多 seed 训练入口为 ``src/ModernTCN/run_modern_tcn_theta10_v2_multiseed.py``。训练脚本读取统一数据集，不重新划分 split，不重新拟合 scaler。

训练脚本记录模型配置、数据契约、损失权重、验证指标、测试指标和 checkpoint 路径。训练输出保存在 ``results/modern_tcn/`` 对应 run_tag 目录。

7.2 多 seed 训练流程

多 seed 训练流程用于比较随机初始化和训练过程对结果的影响。当前默认部署选择 seed 21，run_tag 为 ``modern_tcn_theta10_uniform_h0_v2_seed21``。

多 seed 结果不是软著申请的必要法律事实，但可作为说明书中“对照与验证模块”的实现依据。申请材料应强调软件具备多 seed 训练和结果汇总能力，而不是夸大单个实验结果。

7.3 损失函数和指标

``modern_tcn_metrics.py`` 实现多任务损失和指标计算。指标包括主工况准确率、转向准确率、坡度误差、各类别召回率、过渡窗口表现和用于选模的综合评分。

这些指标用于离线评估和模型选择。闭环控制指标另由 ``src/Compare`` 中的比较脚本计算，二者共同构成“感知性能 + 控制性能”的验证闭环。

7.4 checkpoint 保存

训练脚本保存 PyTorch checkpoint、训练历史 CSV、summary CSV 和训练报告。默认 checkpoint 位于 ``results/modern_tcn/modern_tcn_theta10_uniform_h0_v2_seed21/modern_tcn_seed21.pt``。

checkpoint 属于模型权重文件，不纳入本次源程序鉴别材料。说明书可引用其作为部署模型生成来源。

7.5 ONNX 一致性检查

ONNX 导出后，软件使用 ONNXRuntime 检查 PyTorch 与 ONNX 推理结果的一致性。默认结果目录包含 ``modern_tcn_seed21_onnxruntime_consistency.md`` 和 JSON 记录。

一致性检查用于确保部署文件与训练模型在相同输入下输出一致，降低跨环境部署时的数值偏差风险。

7.6 MATLAB 端一致性检查

MATLAB 端一致性检查由 ``ModernTCN_check_matlab_onnx.m`` 完成。该脚本读取 ONNX 模型和 PyTorch 参考输出，验证 MATLAB 加载器的输出与导出参考是否一致。

通过该检查后，ONNX 模型可由 MATLAB/Simulink 闭环仿真调用。该环节是 Python 训练环境和 MATLAB 控制仿真环境之间的重要接口验证。

第 8 章 Simulink 闭环仿真模块

8.1 闭环模型组成

Simulink 闭环主模型包括 `simulink/LPVMPC\_AGV\_simulink\_Modern\_TCN.slx`、`simulink/LPVMPC\_AGV\_simulink\_GRU.slx`、`simulink/LPVMPC\_AGV\_simulink\_TCN.slx` 和 `simulink/LPVMPC\_AGV\_simulink\_IMU.slx`。其中 ModernTCN 模型是当前主链路，其他模型用于对照。闭环模型通常包含参考路径输入、车辆 S-Function、状态输出、在线分类器、LPV-MPC 控制器、植物模型更新和结果记录模块。

8.2 预加载函数

`preloadfcn\_modern\_tcn.m`、`preloadfcn\_gru.m` 和 `preloadfcn\_tcn.m` 用于在模型加载前准备参数、控制器、路径、数据集、scaler 和预测器。`preloadfcn\_gru.m` 在当前项目中也服务多种模式，不应简单理解为只属于 GRU 旧链路。

预加载函数的存在保证仿真模型打开后具备所需工作区变量，避免手动加载顺序错误造成闭环失败。

8.3 在线预测流程

在线预测流程由 `ModernTCN\_state\_classifier.m`、`ModernTCN\_online\_step.m`、`ModernTCN\_predict\_window.m` 和 `ModernTCN\_load\_predictor.m` 共同实现。模块从车辆输出中提取 19 维特征，维护 128 步窗口，按 scaler 进行归一化，并调用 ONNX/MATLAB 预测器输出三任务结果。在 Simulink 中，`ModernTCN\_State\_Classifier\_sim.m` 作为包装函数连接 MATLAB 在线预测逻辑和仿真模型。

8.4 LPV-MPC 更新流程

LPV-MPC 更新流程由 ModernTCN 输出的调度量、`mpc\_update\_from\_rho.m` 和植物模型更新函数共同完成。调度量经过限幅和变化率控制后用于选择或更新 LPV 模型。

这种设计将感知模型的连续输出纳入控制器调度，但保留 MPC 控制器的约束处理和代价函数结构。

8.5 闭环输出保存

闭环仿真输出包括轨迹、状态、控制输入、工况预测、转向预测、坡度估计、误差指标和分区统计。结果通常保存到 `results/compare/` 下对应实验目录。

`ModernTCN\_analyze\_closed\_loop\_out.m` 和对比脚本负责从仿真输出中提取指标，生成 summary CSV、zones CSV、rank CSV 和 Markdown 报告。

8.6 仿真异常处理

仿真异常可能来自路径文件缺失、控制器缓存缺失、ONNX 文件缺失、`scaler` 不匹配或 Simulink 模型未加载。测试脚本和预加载函数提供了基本检查，用户也可通过 `test_simulink_closed_loop.m` 进行流程验证。

异常处理原则是先确认 `init_project` 已运行，再检查默认配置、路径文件、模型文件、控制器缓存和数据契约是否一致。

第 9 章 对照算法与基线模块

9.1 GRU 基线

GRU 对照模块位于 `src/gru`，包括训练、默认配置、推理、在线分类器和 Simulink 包装函数。当前 GRU 主线使用与 ModernTCN 相同的数据集，便于公平比较。

GRU 在说明书中作为对照与验证模块出现，不作为软件名称中的主创新算法。其作用是帮助评价 ModernTCN 在统一数据和统一闭环平台下的相对表现。

9.2 TCN 基线

TCN 对照模块位于 `src/TCN`，包括训练、多 `seed` 入口、推荐配置、默认配置、预测器加载、窗口预测、在线分类器和 Simulink 配置脚本。

TCN 与 ModernTCN 同属卷积时序模型类别，但结构和训练配置不同。软件保留 TCN 基线用于三算法闭环对比和论文补充验证。

9.3 LPV-MPC theta0 基线

LPV-MPC theta0 基线用于表示不使用 AI 感知坡度调度量的控制链路。该基线有助于评估 ModernTCN 输出相对无感知调度的收益。

基线实验入口为 `run_lpvmpc_theta_baseline_experiment.m`，输出与三算法闭环结果合并比较。

9.4 IMU theta 基线

IMU theta 基线使用简化 IMU 坡度估计作为调度参考。它不是当前主方法，而是用于验证仅依赖简单传感估计时的闭环表现。

说明书中应将该模块描述为可选对照模型，避免将其写成软件主要功能或主要创新。

9.5 true-theta oracle 上界

true-theta oracle 使用真实坡度作为调度参考，代表闭环控制在坡度信息理想可得时的上界参考。它不是实际部署算法，而是评价 ModernTCN 距离理想调度的差距。

该上界用于分析 ModernTCN 在横向跟踪、控制平滑性和坡度调度方面的改进空间。

9.6 对照模块的作用边界

GRU、TCN、theta0、IMU theta 和 oracle 均用于对照验证。申请名称和说明书主体应聚焦 AGV 工况感知、LPV-MPC 调度控制和闭环仿真软件系统，避免把对照算法描述为本软件全部核心。

对照模块的存在增强了软件的验证能力，使软件能够自动输出公平对照结果、多路径结果、扰动结果和实时性结果。

第 10 章 实验、报告与验证模块

10.1 三算法闭环对比

三算法闭环对比由 `compare\_tcn\_gru\_modern\_closed\_loop\_out.m` 实现，比较 ModernTCN、GRU 和 TCN 在同一闭环路径和统一控制平台下的表现。输出包括 summary、zones、rank 和报告文件。

该模块用于评估不同工况感知模型对轨迹跟踪、转向识别、坡度估计和控制输入平滑性的影响。

10.2 多路径闭环实验

多路径闭环实验由 `run\_multi\_path\_closed\_loop\_benchmark.m` 实现。该模块在多条闭环评估路径上运行算法链路，检查结论是否依赖单一路径或单一工况。

多路径评估结果用于增强软件验证流程的鲁棒性和可复现性。

10.3 扰动鲁棒性实验

扰动鲁棒性实验由 `run\_closed\_loop\_robustness\_experiment.m` 实现。该模块在不同扰动等级下运行闭环仿真，评价控制链路对扰动和工况变化的敏感性。

结果可用于比较不同算法在扰动条件下的综合排名、误差变化和控制平滑性。

10.4 实时性测试

实时性测试包括 `benchmark\_modern\_tcn\_onnx\_runtime.py` 和 `run\_realtime\_benchmark.m`。前者测量 ONNXRuntime 单窗口推理时间，后者汇总推理、MPC 求解和整体周期余量。

项目上下文记录 ONNXRuntime+MPC 核心链路 p95 总周期约 0.492 ms，小于 10 ms 控制周期。

MATLAB/Simulink extrinsic 封装主要用于桌面仿真验证，不应表述为最终嵌入式硬实时部署。

10.5 测试脚本

测试脚本位于 `src/tests`，包括 Simulink 闭环测试、GRU 工作流测试、GRU 性能和延迟测试、滤波常数测试、AGV 开环测试和工业开环项目测试。

这些测试用于验证项目初始化、模型加载、仿真链路、分类器推理和性能统计的可用性。

10.6 输出报告

软件自动输出 Markdown、CSV、MAT、PNG/PDF 图表和模型文件。软著源码材料仅纳入 MATLAB/Python 源程序；结果报告和图表只作为说明书中的功能输出示例，不纳入源程序鉴别材料。

报告文件使每次训练和闭环实验具备可追溯性，便于复核输入数据、模型版本、路径和指标。

第 11 章 用户使用说明

11.1 初始化项目

用户在 MATLAB 中进入项目根目录后运行 `init_project`，该函数将 `src/core`、`src/lpv`、`src/mpc`、`src/paths`、`src/gru`、`src/TCN`、`src/ModernTCN` 和 `src/Compare` 加入搜索路径。

`project_root.m` 用于定位项目根目录，`results_dir.m` 用于生成或返回结果目录。

11.2 加载默认 ModernTCN 配置

默认配置由 `ModernTCN_default_config(project_root())` 返回。配置包含 `seed`、`run_tag`、数据集文件、`raw train data`、ONNX 文件、参考输出文件和部署侧 `theta` 调理参数。

用户若临时测试其它 checkpoint，可在调用脚本中覆盖 `run_tag`、`onnx_file` 或 `dataset_file`，但提交材料中应保持默认版本描述一致。

11.3 运行离线训练

离线训练通过 Python 执行。用户可运行 `train_modern_tcn.py` 进行单 `seed` 训练，也可运行 `run_modern_tcn_theta10_v2_multiseed.py` 进行多 `seed` 实验。

训练前应确认数据集和数据契约存在，且 Python 环境安装了 PyTorch 等依赖。

11.4 导出 ONNX 模型

训练完成后运行 `export_modern_tcn_onnx.py` 导出 ONNX。导出脚本使用固定输入形状 `[1,128,19]`，输出名称为 `logits_main`、`logits_turn`、`theta_hat`。

导出后建议运行 ONNXRuntime 一致性检查和 MATLAB ONNX 一致性检查。

11.5 运行闭环仿真

用户加载 `simulink/LPVMPC_AGV_simulink_Modern_TCN.slx` 后运行仿真。仿真前应确保预加载函数能够找到控制器、LPV 网格、路径、数据集、`scaler` 和 ONNX 文件。

若需要批量运行，可使用 `run_closed_loop_model_once.m` 或 `src/Compare` 下的实验入口脚本。

11.6 运行对比实验

对比实验包括 ModernTCN/GRU 对比、ModernTCN/GRU/TCN 三算法对比、LPV-MPC theta 基线、多路径闭环、扰动鲁棒性和实时性测试。

这些脚本输出统一格式的指标表和报告，便于申请人或开发者复核软件功能。

11.7 查看结果

训练结果位于 ``results/modern_tcn/``、``results/gru/`` 和 ``results/tcn/``。闭环对比结果位于 ``results/compare/``。论文图表和补充结果位于 ``results/paper/``。

用户查看结果时应注意区分源码、数据、模型、结果和论文图表，不应将结果文件误作为源程序材料提交。

11.8 常见问题

若 MATLAB 提示找不到函数，应先运行 ``init_project``。若 ONNX 文件缺失，应检查 ``ModernTCN_default_config.m`` 的 `run_tag` 和路径。若维度不匹配，应检查数据契约和 `scaler` 是否对应当前数据集。

若 Simulink 闭环加载失败，应检查 ``data/models`` 下控制器和线性化数据库是否存在，以及 Simulink 模型是否与预加载函数匹配。

第 12 章 技术特点

12.1 面向 AGV 闭环控制的多任务时序感知

软件将主工况分类、转向方向分类和坡度回归组织为同一 ModernTCN 多任务模型，避免为每个任务维护完全独立的数据链和部署链。

该设计使在线闭环能够同时获得离散状态和连续调度量，适用于坡道、转向和异常工况共同存在的 AGV 场景。

12.2 LPV-MPC 与深度时序模型协同

软件保留 LPV-MPC 的控制器结构和约束处理能力，同时利用 ModernTCN 输出辅助调度。深度模型负责感知，MPC 负责控制优化，两者在闭环中协同工作。

这种模块化关系使软件既能接入 ModernTCN，也能接入 GRU、TCN 或其它对照模型进行比较。

12.3 统一数据契约与公平对照

当前 ModernTCN、GRU 和 TCN 使用同一数据集、同一 19 维特征、同一 run-level split 和同一 scaler 策略。该设计降低了数据差异对算法比较的干扰。

数据契约文件提供了可追溯的输入、标签、split 和 scaler 定义，便于后续复现实验。

12.4 ONNX/MATLAB/Simulink 跨环境部署

软件支持从 PyTorch 训练模型导出 ONNX，再由 MATLAB 端加载并在 Simulink 闭环中调用。

ONNXRuntime 和 MATLAB 一致性检查用于验证跨环境输出一致性。

该链路提升了模型从训练环境到控制仿真环境的可迁移性。

12.5 多路径、扰动和实时性验证

软件提供多路径闭环实验、扰动鲁棒性实验和实时性测试，能够从不同角度评价闭环控制链路。验证结果以 CSV 和 Markdown 报告形式保存。

这些验证模块使软件不仅能训练模型，还能系统评估模型对控制性能的影响。

第 13 章 数据安全与维护

13.1 本地数据处理

软件默认在本地文件系统处理数据、模型和结果，不需要将训练数据或身份材料上传到外部服务。申请材料生成时也不写入身份证、营业执照、公章或签名扫描件。

如申请人另行公开仓库，应自行确认公开内容是否包含敏感信息、第三方数据或权属受限材料。

13.2 文件管理

项目通过 `PROJECT\_FLOW\_MANIFEST.md` 区分 KEEP_ACTIVE、KEEP_RESULT、ARCHIVE_LEGACY、DELETE_CANDIDATE 和 REVIEW 文件。软著材料生成时遵循白名单源码选择和排除规则。

建议在提交材料前保持 `soft\_copyright\_application/` 与源码仓库分离备份，避免后续实验结果覆盖申请材料。

13.3 结果可追溯

训练脚本、闭环脚本和报告文件记录数据集、模型配置、seed、路径和指标。默认 ModernTCN 部署模型固定为 seed 21，便于追溯。

可追溯性有助于确认申请材料中的功能描述与项目实际文件一致。

13.4 版本维护

当前材料对应 V1.0。若后续修改软件名称、数据契约、模型结构、默认 seed、Simulink 模型或控制器配置，应同步更新说明书、申请字段、源码页眉和一致性检查报告。

版本维护时应保留变更记录，避免不同材料中出现软件名称、版本号或模型路径不一致。

第 14 章 版本说明

14.1 V1.0 功能范围

V1.0 包括项目初始化、AGV 车辆模型、LPV 线性化、MPC 控制器、路径生成、统一数据集、ModernTCN 训练部署、MATLAB 在线推理、Simulink 闭环仿真、GRU/TCN 对照、多路径与扰动鲁棒性实验、实时性测试和材料一致性检查。

V1.0 不声明已经完成嵌入式硬实时部署，不包含第三方框架源码，不包含申请人身份材料和权属证明文件。

14.2 后续扩展方向

后续可扩展方向包括嵌入式部署接口、更多传感器融合、更多路径和载荷工况、在线异常诊断、控制器参数自动整定、结果可视化界面和更严格的自动化测试。

这些扩展不影响当前 V1.0 作为 AGV 工况感知与 LPV-MPC 闭环控制仿真软件的功能边界。

附录 A 主要源码文件清单

模块	主要文件	说明
项目初始化	init_project.m, project_root.m, results_dir.m	设置路径和结果目录
车辆模型	src/core/agv_model_sfunc.m, state_eq.m, output_eq.m	AGV 动力学与输出
LPV 线性化	src/lpv/lin_agv_at_point.m, lin_agv_grid.m	生成线性化模型网格
MPC 控制	src/mpc/mpc_setup_single_interp.m, mpc_update_from_rho.m, Cost_Function.m	构造与更新控制器
路径生成	src/paths/*.m	训练、展示和评估路径
ModernTCN	src/ModernTCN/*.py, src/ModernTCN/*.m	训练、导出、MATLAB 在线部署
对照算法	src/gru, src/TCN, src/Compare	GRU/TCN 基线和闭环比较
测试验证	src/tests/*.m	初始化、开环、闭环和性能测试

附录 B 主要输入输出文件清单

类型	文件	用途
数据契约	data/tcn/ModernTCN_dataset_agv_dualsteer_theta10_uniform_conf_h0_v2_contract.json	定义特征、标签、split 和 scaler
统一数据集	data/tcn/ModernTCN_dataset_agv_dualsteer_theta10_uniform_conf_h0_v2.mat	ModernTCN/GRU/TCN 训练测试

类型	文件	用途
ONNX 模型	results/modern_tcn/modern_tcn_theta10_uniform_h0_v2_seed21/modern_tcn_seed21.onnx	MATLAB/Simulink 部署
ModernTCN 模型	results/modern_tcn/modern_tcn_theta10_uniform_h0_v2_seed21/modern_tcn_seed21.pt	PyTorch checkpoint
Simulink 模型	simulink/LPVMPC_AGV_simulink_Modern_TCN.slx	ModernTCN 闭环
对比结果	results/compare/	闭环指标、鲁棒性和实时性报告

附录 C 关键参数表

参数	当前值	说明
vehicle_type	diagonal_dual_steer_drive_agv	对角双转向驱动 AGV
active_drive_steer_wheels	LF, RR	主动驱动/转向轮
passive_support_wheels	RF, LR	被动支撑轮
Ts	0.01 s	控制采样周期
seq_len	128	输入窗口长度
input_dim	19	输入特征维度
run_tag	modern_tcn_theta10_uniform_h0_v2_seed21	默认部署模型标签
channels	64	ModernTCN 通道数
blocks	5	ModernTCN 残差块数
kernel_size	31	大核时序卷积核
temporal_padding	same	默认时序填充方式
dropout	0.15	dropout 比例
expansion	2	通道扩展倍率

附录 D 术语表

术语	含义
AGV	Automated Guided Vehicle，自动导引车
LPV	Linear Parameter-Varying，线性参数变化模型
MPC	Model Predictive Control，模型预测控制
ModernTCN	本软件使用的大核时序卷积多任务感知模型
ONNX	Open Neural Network Exchange，跨框架模型交换

术语	含义
	格式
scaler	训练集拟合、验证/测试/在线应用的归一化参数
logits	分类模型 softmax 前的输出
theta_hat	坡度或调度量回归输出